Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №4

по курсу «Численные методы»

«Решение систем нелинейных уравнений методом Ньютона»

Студент группы ИУ9-62Б: Трофименко Д. И.

Преподаватель: Домрачева А. Б.

Москва 2026

1 Постановка задачи

1.1 Дано:

Дана система нелинейных уравнений

$$\begin{cases} f_1(x_1, x_2, \ldots, x_n) = 0, \\ f_2(x_1, x_2, \ldots, x_n) = 0, \\ \ldots \\ f_n(x_1, x_2, \ldots, x_n) = 0. \end{cases}$$

1.2 Найти:

- Решение системы графически и принять полученное решение за начальное приближение
- Уточнить решение методом Ньютона с точностью

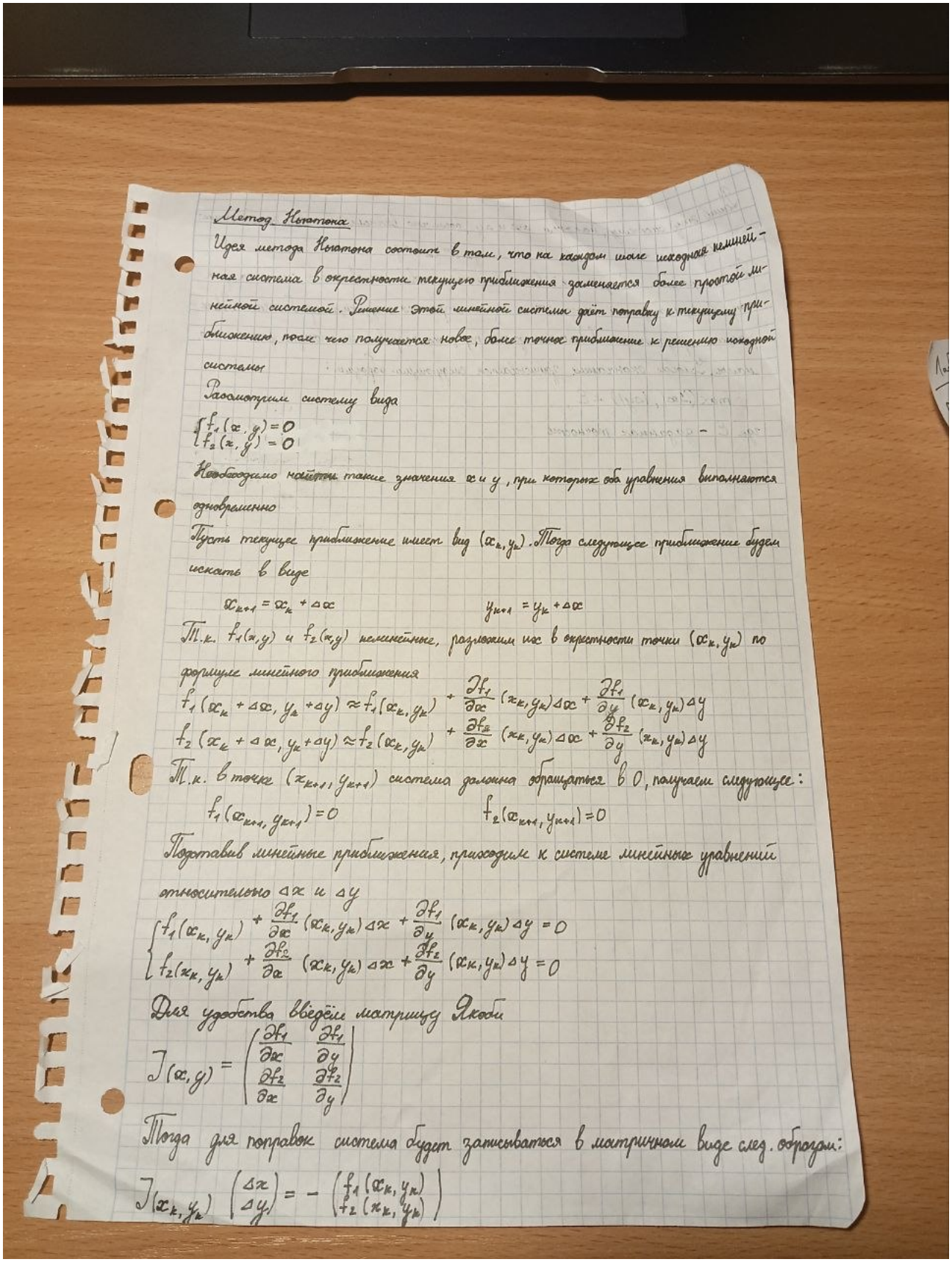
$$\varepsilon = 0.01.$$

1.3 Индивидуальный вариант:

В качестве индивидуального варианта была взята следующая система:

$$\begin{cases} \sin(y+1) - x = 1.2, \\ 2y + \cos x = 2 \end{cases}$$

2 Теоретические сведения



Метод Ньютона

Идея метода Ньютона состоит в том, что на каждом шаге исходная нелинейная система в окрестности текущего приближения заменяется более простой линейной системой. Решение этой линейной системы даёт поправку к текущему приближению, после чего получается новое, более точное приближение к решению исходной системы

Рассмотрим систему вида

$$\begin{cases} f_1(x, y) = 0 \\ f_2(x, y) = 0 \end{cases}$$

Необходимо найти такие значения x и y, при которых оба уравнения выполняются одновременно

Пусть текущее приближение имеет вид (x_k, y_k). Тогда следующее приближение будем искать в виде

$$x_{k+1} = x_k + \Delta x \qquad y_{k+1} = y_k + \Delta x$$

Т.к. $f_1(x,y)$ и $f_2(x,y)$ нелинейные, разложим их в окрестности точки (x_k, y_k) по формуле линейного приближения

$$f_1(x_k + \Delta x, y_k + \Delta y) \approx f_1(x_k, y_k) + \frac{\partial f_1}{\partial x}(x_k, y_k)\Delta x + \frac{\partial f_1}{\partial y}(x_k, y_k)\Delta y$$

$$f_2(x_k + \Delta x, y_k + \Delta y) \approx f_2(x_k, y_k) + \frac{\partial f_2}{\partial x}(x_k, y_k)\Delta x + \frac{\partial f_2}{\partial y}(x_k, y_k)\Delta y$$

Т.к. в точке (x_{k+1}, y_{k+1}) система должна обращаться в 0, получаем следующее:

$$f_1(x_{k+1}, y_{k+1}) = 0 \qquad f_2(x_{k+1}, y_{k+1}) = 0$$

Подставив линейные приближения, приходим к системе линейных уравнений относительно Δx и Δy

$$\begin{cases} f_1(x_k, y_k) + \frac{\partial f_1}{\partial x}(x_k, y_k)\Delta x + \frac{\partial f_1}{\partial y}(x_k, y_k)\Delta y = 0 \\ f_2(x_k, y_k) + \frac{\partial f_2}{\partial x}(x_k, y_k)\Delta x + \frac{\partial f_2}{\partial y}(x_k, y_k)\Delta y = 0 \end{cases}$$

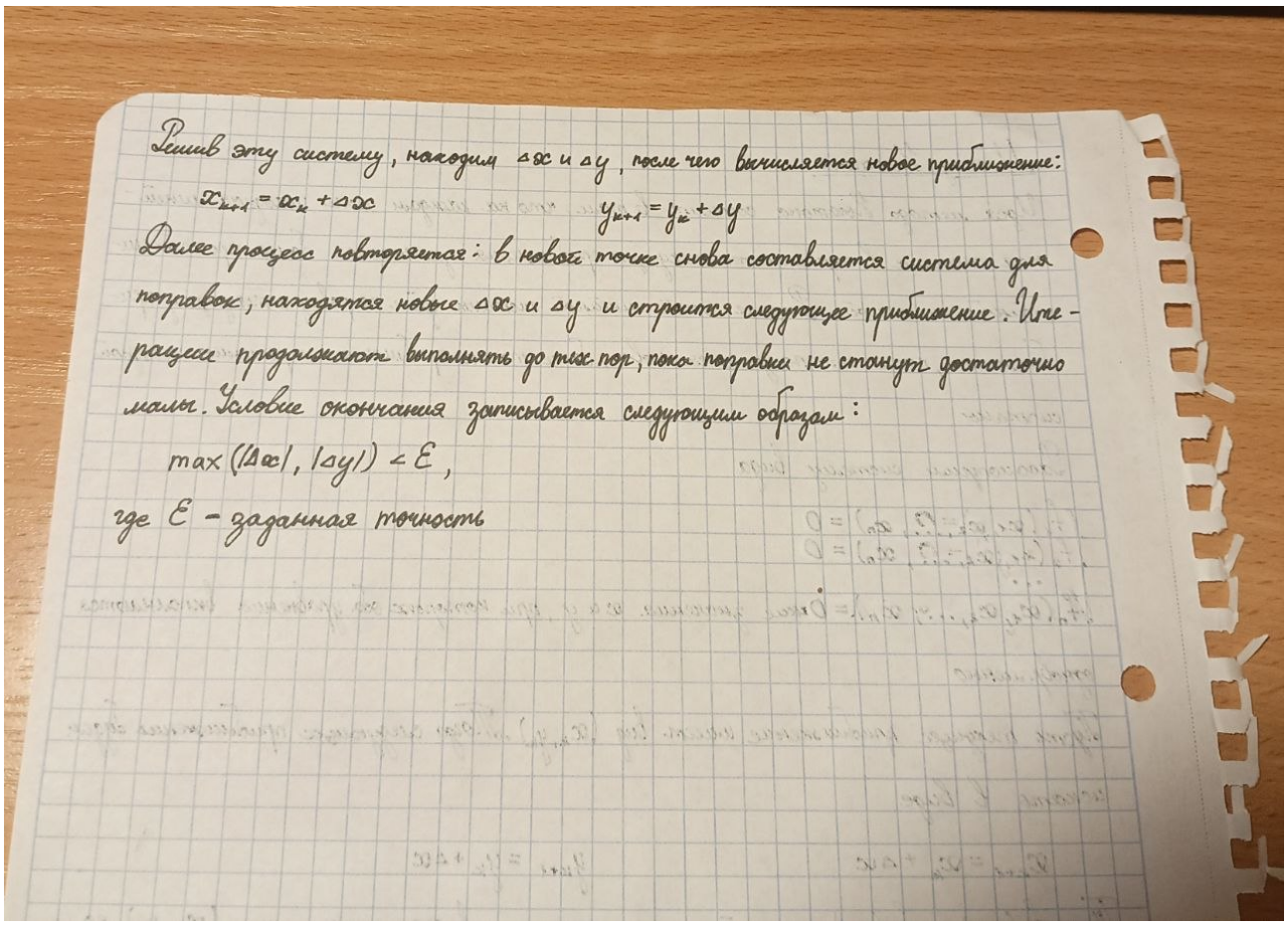
Для удобства введём матрицу Якоби

$$J(x, y) = \begin{pmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{pmatrix}$$

Тогда для поправок система будет записываться в матричном виде след. образом:

$$J(x_k, y_k) \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = - \begin{pmatrix} f_1(x_k, y_k) \\ f_2(x_k, y_k) \end{pmatrix}$$

Рис. 1 — Теоретические сведения (1 часть)



Решив эту систему, находим Δx и Δy, после чего вычисляется новое приближение:

$$x_{k+1} = x_k + \Delta x \qquad y_{k+1} = y_k + \Delta y$$

Далее процесс повторяется: в новой точке снова составляется система для поправок, находятся новые Δx и Δy и строится следующее приближение. Итерации продолжают выполнять до тех пор, пока поправки не станут достаточно малы. Условие окончания записывается следующим образом:

$$\max(|\Delta x|, |\Delta y|) < \varepsilon,$$

где ε – заданная точность

Рис. 2 — Теоретические сведения (2 часть)

3 Практическая реализация

Исходный код программы представлен в листингах 1.

Листинг 1 — lab4.py.

```
import math
import numpy as np

EPS = 0.01

X0 = -0.20184
Y0 = 0.51015

def f(x, y):
    return np.array([
        math.sin(y + 1) - x - 1.2,
        2 * y + math.cos(x) - 2
    ], dtype=float)

def jacobian(x, y):
    return np.array([
        [-1.0, math.cos(y + 1)],
        [-math.sin(x), 2.0]
    ], dtype=float)

def newton(x0, y0, eps=EPS, max_iter=100):
    x = x0
    y = y0
    for _ in range(max_iter):
        delta = np.linalg.solve(jacobian(x, y), -f(x, y))
        x_new = x + delta[0]
        y_new = y + delta[1]
        if max(abs(x_new - x), abs(y_new - y)) < eps:
            return x_new, y_new
        x, y = x_new, y_new
    return x, y

def main():
    x, y = newton(X0, Y0)
    print("Solution obtained graphically:")
    print(f"x = {X0:.5f}, y = {Y0:.5f}")
    print()
    print("Solution by Newton's method:")
    print(f"x = {x:.10f}, y = {y:.10f}")

if __name__ == "__main__":
    main()
```

3.1 Результаты

В качестве исходных данных задаются следующие значения:

- первая функция системы нелинейных уравнений, записанная в виде $f_1(x, y) = 0$: $f_1(x, y) = \sin(y + 1) - x - 1.2$
- вторая функция системы нелинейных уравнений, записанная в виде $f_2(x, y) = 0$: $f_2(x, y) = 2y + \cos x - 2$
- заданная точность вычислений: $\varepsilon = 0.01$

3.2 Вывод программы:

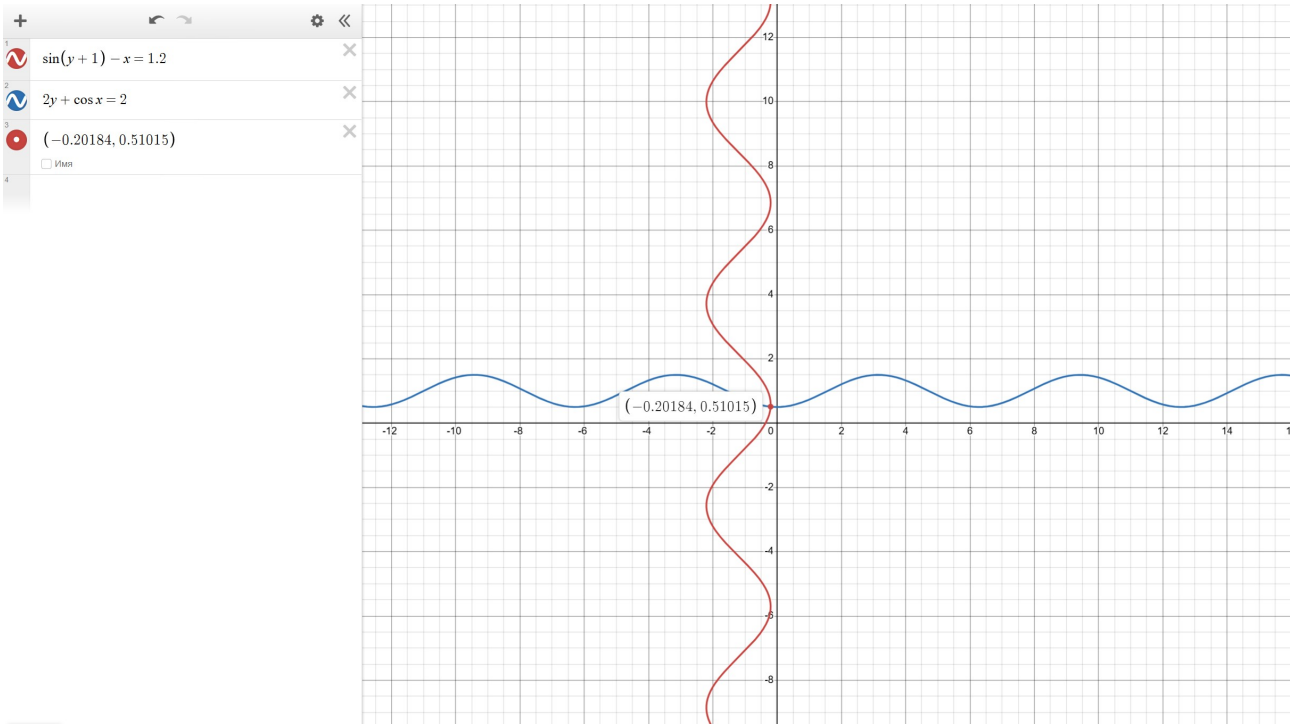


Рис. 3 — Графическое решение исходной системы

```
Решение, полученное графически:
x = -0.20184, y = 0.51015

Решение методом Ньютона:
x = -0.2018384154, y = 0.5101501575
```

Рис. 4 — Вывод программы в консоль

4 Выводы

В ходе выполнения лабораторной работы был изучен и реализован метод Ньютона для решения системы нелинейных уравнений. Были рассмотрены его теоретические основы и порядок применения к системе из двух уравнений. В процессе работы система была приведена к удобному виду, выбрано начальное приближение по графическому решению и выполнено последующее уточнение итерационным способом.

По результатам тестирования было установлено, что метод даёт точное решение и хорошо уточняет начальное приближение. Об этом говорит то, что найденное после итераций решение практически совпадает с графически определённой точкой пересечения, но при этом является более точным. Небольшое изменение по сравнению с исходным приближением показывает, что графическое решение было выбрано удачно, а сам метод корректно выполнил уточнение.

Также можно сделать вывод о высокой эффективности и скорости работы метода. Решение было получено очень быстро, поскольку начальное приближение находилось близко к истинному корню системы. Благодаря этому потребовалось малое число итераций для достижения заданной точности.